

Tool-Agnosticism: The Three Minimal Requirements for a CKS Substrate Host

Author: Wenxin Li (Independent Researcher)

ORCID: 0009-0004-8065-3235

Date: 1 May 2026

License: Creative Commons Attribution 4.0 International (CC BY 4.0)

Attribution

This work derives from and formalizes the Coordination Knowledge Substrate (CKS) pattern introduced in *Coordination Outside the Model: A Human-Governed Substrate Pattern for AI Systems* (Li, April 2026). It does not introduce new axioms. Its sole contribution is to articulate, in operational form, the precise content of the source paper’s tool-agnosticism commitment, so that downstream work can adopt or argue against the term without ambiguity.

Abstract

The Coordination Knowledge Substrate (CKS) design pattern names “tool-agnostic” as one of its six architectural commitments. The commitment is most often misread in a marketing sense — that the pattern “runs anywhere” — when its actual architectural content is a precise interface specification: any environment satisfying three named requirements can host a CKS substrate, and no environment failing any one of them can. The three requirements (persistent structured state, human read/write access, LLM access to substrate content) are jointly sufficient and individually necessary. This note formalizes them precisely, distinguishes tool-agnosticism from three adjacent commitments (model-agnosticism, vendor-neutrality, open-source mandates), states what the commitment makes architecturally possible — non-specialist governance most centrally — identifies the environments that fail the requirements, and provides an operational test for whether a given environment qualifies as a CKS substrate host.

1. Why tool-agnosticism needs to be stated precisely

The CKS pattern uses “tool-agnostic” as the architectural commitment that closes the substrate-layer accessibility argument the source paper develops in §7.1. The commitment is what makes the pattern’s other commitments — substrate as coordination artifact, conflict preservation, human governance, AI as substrate mediator, linear-cost scaling — implementable in commodity infrastructure rather than only in specialized governance runtimes, and what underwrites the architectural property of non-specialist governance (§7.4): the rights humans hold over substrate content and orchestration rules can only be exercised in environments humans actually use, and naming which environments qualify is the work tool-agnosticism does.

The commitment is not self-defining. The phrase “tool-agnostic” is structurally similar to adjacent claims widely made in the 2024–2026 AI infrastructure landscape — “platform-agnostic,” “protocol-agnostic,” “model-agnostic,” “vendor-neutral” — and the surface similarity invites reading CKS’s tool-agnosticism as one of these. It is not. Each of those neighbors describes agnosticism at a particular layer, often inside a specialized runtime; CKS’s tool-agnosticism describes agnosticism at the

substrate-host layer. The source paper at §7.1 develops the contrast directly against Reign, the protocol-agnostic agentic governance platform whose protocol-agnosticism describes its ability to sit in front of multiple agent and tool protocols *inside its specialized runtime*. CKS’s substrate, by contrast, can be hosted in environments without any specialized runtime at all — not because CKS is more agnostic than Reign, but because the two are agnostic at different layers.

The misreading the precise statement preempts is the inversion of this contrast: that any environment a vendor describes as “agnostic” qualifies as a CKS host, even when the environment requires a specialized runtime, a specific schema, or a specific governance API. A specialized AI platform that adds requirements beyond the three named in §2 is not what the architecture means by tool-agnostic, even if its internal layers are themselves agnostic in some other sense; a spreadsheet that satisfies the three requirements with no specialized runtime at all is. Any binding to a specific host beyond the three requirements introduces a dependency the architecture does not have.

2. The three minimal requirements

The source paper at §7.1 states the commitment as follows: any environment meeting three minimal requirements — persistent structured state, human read/write access, and LLM access to substrate content — can instantiate the CKS pattern. The three requirements are stated here in operational form.

Requirement 1 — Persistent structured state. The host environment must support content that persists across reads and writes, with structure the substrate can rely on. *Structure* here means addressable units — entities, relationships, fields, rows, records, or equivalent — such that a particular piece of substrate content can be identified, located, and acted on as a unit. The structure does not need to conform to any specific schema; the substrate’s schema is a property of the cell, not of the host. The host must provide the addressability and the persistence; the substrate provides the schema and the content. Ephemeral storage — per-session context windows, transient caches, in-memory state that does not survive restart — does not satisfy this requirement.

Requirement 2 — Human read/write access. Humans must be able to read and write substrate content directly, without intermediation by an LLM or by a specialized runtime layer. *Directly* means the path from human to substrate does not pass through an LLM call, an embedding query, or a runtime gateway as a precondition of access — the human inspects and modifies substrate content in inspectable form. The interface form is unconstrained: a spreadsheet, a database client, a document editor, a wiki, or a structured-text file are all valid. The host must provide the directness; the form the directness takes is a deployment decision.

Requirement 3 — LLM access to substrate content. The LLM used as substrate mediator must be able to read substrate content as input to its operations and write substrate content as output. Read access supplies the LLM with the prior decisions, rationale, and preserved conflicts the substrate carries; write access lets the LLM draft and update substrate content under human governance and human-authored orchestration rules. The access mechanism — direct API call, file read, query, retrieval — is unconstrained; what the host must provide is that the access is possible at all, in both directions.

Joint sufficiency, individual necessity. The three requirements are jointly sufficient and individually necessary. *Sufficient*: an environment that satisfies all three can host a CKS substrate; no further environmental capability is required, and any further capability the environment provides may be useful

but is not architecturally needed. *Necessary*: an environment failing any one of the three cannot host a CKS substrate, regardless of what else it provides. An environment that adds a fourth requirement — a specific schema, a specific runtime, a specific governance API — may be useful for some other architecture, but is no longer the kind of host CKS’s tool-agnosticism commits to, because the architecture’s claim is that the three requirements are *enough*.

3. What tool-agnosticism is NOT

The definition in §2 is precise about what tool-agnosticism *is*. Three adjacent commitments are commonly conflated with it; each is a real and reasonable position in some architecture, and each says something different from what tool-agnosticism says.

Not model-agnosticism. Model-agnosticism is the commitment that a system works with any LLM, regardless of vendor or capability class. CKS’s tool-agnosticism *implies* model-agnosticism in the sense that Requirement 3 names “the LLM used as substrate mediator” without specifying which one — any LLM that can read and write substrate content satisfies the requirement. But model-agnosticism is a distinct commitment about the *mediator*, not about the *host*. A system can be tool-agnostic while preferring a specific LLM at the mediator level, and a system can be model-agnostic in its choice of LLM while running on a host that fails one of the three substrate requirements. The two commitments operate on different objects.

Not vendor-neutrality. Vendor-neutrality is a commercial commitment that a system avoids lock-in to a specific vendor. Tool-agnosticism enables vendor-neutrality — a substrate hosted in any environment satisfying the three requirements can in principle be migrated to any other environment that does — but does not require it. A deployment that runs entirely on a single vendor’s stack can still satisfy CKS’s tool-agnosticism, provided the substrate’s host environment satisfies the three requirements and could be migrated to another that also does. Vendor-neutrality is a property of a procurement decision; tool-agnosticism is a property of the architecture’s host interface.

Not an open-source mandate. Tool-agnosticism makes no claim about the licensing of the host environment. A proprietary spreadsheet, a proprietary database, a proprietary document store, and an open-source equivalent of any of these can each satisfy the three requirements; the architecture treats them as equivalent at the host-interface layer. Whether to prefer open-source hosts on other grounds — auditability, cost, ecosystem fit, regulatory posture — is a deployment decision the architecture does not weigh in on.

4. What tool-agnosticism makes possible

Tool-agnosticism is not an end in itself; it is the architectural property that allows several other commitments of the CKS pattern to take the form they take. Four downstream consequences follow.

Non-specialist governance (§7.4). This is the most consequential. The CKS pattern commits to non-specialist governance as an architectural property: a non-specialist must be able to inspect, modify, and override substrate content and orchestration rules in tools they already use. That commitment is realizable only because the three requirements are met by commodity tools — spreadsheets, document editors, wikis, project-tracking systems with structured fields — rather than by specialized governance runtimes. If either Requirement 1 or Requirement 2 could only be satisfied by a specialized runtime, non-specialist governance would collapse to specialist governance behind a friendlier interface.

Bottom-up adoption. Hosting a CKS substrate does not require infrastructure investment beyond what the three requirements specify. A team with access to a host environment satisfying the three requirements can begin using the pattern at the cell level without organizational sign-off on a platform, without procurement of a specialized runtime, and without coordination with adjacent teams as a precondition. Adoption can grow from individual cells to multi-cell coordination as the pattern proves out.

Migration safety. Because the substrate’s host interface is three minimal requirements rather than a specific API, a substrate can be migrated between hosts that satisfy the requirements without the substrate’s content or orchestration rules changing form. Migration correctness depends on the determinism contract the architecture commits to — substrate content is the same regardless of host — but the architectural permission to migrate at all comes from tool-agnosticism. A pattern bound to a specific host cannot be migrated; a pattern bound to a host *interface* can.

Composability with adjacent systems. Because the host is not a specialized runtime, a CKS substrate can coexist with other systems in the same environment — databases, document stores, version control, retrieval indices — without those systems needing to be aware of the CKS pattern. The host environment is the shared surface on which adjacent systems and the substrate operate on their own terms. This is what allows CKS to be deployed alongside existing infrastructure rather than in place of it.

5. What environments fail the requirements

Stating the failure modes explicitly is what makes the boundary auditable. Each of the following is a real architectural choice in 2024–2026 AI infrastructure, and each fails at least one of the three requirements as a substrate host.

Pure context-window memory. Holding “substrate” content inside an LLM’s context window across turns or sessions does not satisfy Requirement 1 — the content does not persist across sessions in addressable structured form — and fails Requirement 2 because the content is not directly human-readable as state outside of model invocation.

Black-box agent memory. Memory held inside an agent framework and accessed only through the agent’s API does not satisfy Requirement 2: the human cannot read or write the memory directly as substrate state, only through the agent’s mediation. The architectural commitment to direct human access does not survive the API gate.

LLM-only stores. A vector database or a knowledge graph that is only readable through embedding queries or graph traversals issued by an LLM does not satisfy Requirement 2, because the store’s content is not directly human-readable in inspectable form. A human inspecting an embedding does not inspect substrate content; they inspect a numerical artifact of substrate content. The same limitation applies whenever the only access path to a store passes through machine-mediated retrieval.

Specialized runtimes that gate access. An environment that requires substrate access to flow through a specific governance API, workflow engine, or approval gateway as a precondition of read or write does not satisfy Requirement 2 if the gating prevents direct human read/write at any point. Gating that operates as a layer over a directly-accessible substrate (audit logs, observation hooks, optional review queues) is admissible; gating that operates as a precondition of access is not. The distinction is whether

the gate sits beside the substrate or in front of it.

A system that uses any of the above as part of its overall architecture can still be CKS-coherent if those components are layered on top of a substrate that does satisfy the three requirements — for example, a vector index built over a human-readable substrate, with the index as a derived artifact rather than the authoritative store. The substrate itself must satisfy the requirements; auxiliary components built around or over it need not.

6. Operational test

An environment can host a CKS substrate if and only if all of the following are true at all times during the substrate’s existence:

1. Substrate content persists across reads and writes in addressable structured form, surviving session boundaries and process restarts.
2. A human with appropriate access can read and write substrate content directly, in inspectable form, without LLM or specialized-runtime intermediation as a precondition of access.
3. The LLM used as substrate mediator can read substrate content as input to its operations and write substrate content as output, through whatever mechanism the host provides.
4. No additional environmental capability is required for the substrate to function. Additional capabilities the environment provides may be used; they must not be required as a precondition of substrate operation.
5. The above requirements continue to be satisfied if the substrate is migrated to another environment that also satisfies them — the requirements describe the host *interface*, not a host’s particular implementation.

A host that fails any of (1)–(5) may be a useful host for some other architecture, and a substrate running on it may be useful in some other sense, but is not a CKS substrate host as the architecture defines the term.

7. Why naming this commitment matters

Tool-agnosticism is the architectural property that prevents the CKS pattern from collapsing into “CKS-on-platform-X.” Implementations that bind the substrate to a specific host beyond the three requirements may be useful for that host and may be the right deployment choice in many circumstances; what they cannot do is preserve the architectural claim that CKS substrates are interoperable across hosts that satisfy the three requirements. The interoperability claim is what underwrites bottom-up adoption, migration safety, and non-specialist governance; binding the substrate to a specific host removes the architectural basis for all three at once.

Naming the three requirements precisely is what makes the cross-host claim defensible. A CKS substrate hosted in a spreadsheet, a CKS substrate hosted in a structured document store, and a CKS substrate hosted in a relational database are the same architectural object, because they meet the same host-interface specification; they are not three different patterns sharing a name. Subsequent work that adopts the CKS pattern, extends it, composes it with adjacent patterns, or argues against it should use “tool-agnostic” in the sense formalized here. Subsequent work that uses the term to mean something else — agnosticism inside a specialized runtime, vendor-neutrality, model-agnosticism, or any

combination of these — is using a different concept, and the difference should be named.

Source paper

Li, W. (2026). *Coordination Outside the Model: A Human-Governed Substrate Pattern for AI Systems*. April 2026. ORCID: 0009-0004-8065-3235.

How to cite this note

Li, W. (2026). *Tool-Agnosticism: The Three Minimal Requirements for a CKS Substrate Host*. 1 May 2026. ORCID: 0009-0004-8065-3235.